

Exercício 16 – Geração de Arquivo JSON a partir do BHCF012S

1. Ficha Técnica

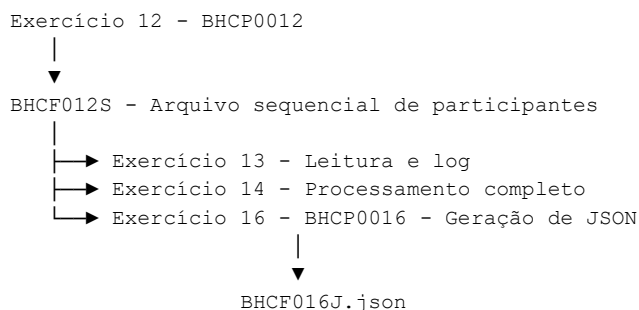
Item	Informação
Exercício	16
Título	Geração de Arquivo JSON a partir de Arquivo Sequencial
Programa	BHCP0016
Entrada	BHCF012S – Arquivo sequencial de participantes
Saída	BHCF016J – Arquivo JSON de participantes

2. Objetivo

Desenvolver um programa COBOL Batch capaz de ler o arquivo sequencial BHCF012S, gerado no Exercício 12, e criar um arquivo JSON contendo a lista de participantes do Bootcamp Hackathon COBOL.

O programa deverá montar manualmente a estrutura JSON usando comandos tradicionais de COBOL, sem utilizar EXEC JSON, JSON GENERATE ou comandos nativos de manipulação JSON.

3. Contexto na Cadeia do Mini Sistema Batch



O Exercício 16 demonstra que um arquivo posicional tradicional de COBOL pode ser transformado em um formato moderno de integração, como JSON.

4. Conceito Didático

JSON é texto estruturado. Se o COBOL consegue montar texto e gravar arquivo sequencial, então consegue gerar um arquivo JSON manualmente.

COBOL tradicional	JSON
Arquivo posicional	Objeto com pares nome/valor
Campo PIC X/9	Campo JSON string ou number
OCCURS	Array JSON
WRITE em arquivo sequencial	Gravação de linhas JSON
Layout por posição	Estrutura por nome de campo

5. Arquivos do Exercício

Tipo	DDNAME	Arquivo físico sugerido	Descrição
Entrada	BHCF012S	BHCF012S.txt	Arquivo sequencial gerado no Exercício 12
Saída	BHCF016J	BHCF016J.json	Arquivo JSON com participantes

6. Layout de Entrada – BHCF012S

Campo	PIC	Tamanho	Posição	Descrição
FD-CODIGO	9(005)	5	1–5	Código do participante
FD-NOME	X(030)	30	6–35	Nome do participante
FD-UF	X(002)	2	36–37	Unidade federativa
FD-TRILHA	X(010)	10	38–47	Trilha do Bootcamp
FD-SITUACAO	X(010)	10	48–57	Situação do participante
FD-DATA	9(008)	8	58–65	Data no formato AAAAMMDD

Tamanho total do registro: 65 bytes.

7. Massa de Entrada Esperada

00001	ANA SILVA	SPCOBOL	ATIVO	20260701
00002	BRUNO LIMA	RJCOBOL	ATIVO	20260701
00003		MGJCL	ATIVO	20260701
00004	DANIEL COSTA	BACOBOL	ATIVO	20260701
00005	ELISA ROCHA	DB2	ATIVO	20260701
00006	FERNANDO ALVES	SCVSAM	ATIVO	20260701
00007	GABRIELA NUNES	RSCOBOL	ATIVO	20260701
00008	HENRIQUE DIAS	PEJCL	ATIVO	20260701
00009	ISABELA MOURA	CEDB2	ATIVO	20260701
00010	JOAO PEREIRA	GOVSAM	ATIVO	20260701

8. Regra sobre Registros com Campos em Branco

O Exercício 16 deverá gerar JSON com todos os 10 registros, inclusive os registros 00003 e 00005, preservando campos em branco como string vazia.

Registro	Campo em branco	Representação no JSON
00003	Nome	"nome": ""
00005	UF	"uf": ""

Neste exercício, o foco é transformação de formato, não validação de regra de negócio.

9. Estrutura JSON Esperada

```
{
  "participantes": [
    {
      "codigo": "00001",
      "nome": "ANA SILVA",
```

```

        "uf": "SP",
        "trilha": "COBOL",
        "situacao": "ATIVO",
        "data": "20260701"
    }
]
}

```

10. Práticas Técnicas

- OPEN INPUT e OPEN OUTPUT
- READ com AT END
- WRITE em arquivo LINE SEQUENTIAL
- FILE STATUS
- PERFORM UNTIL
- Montagem de texto com STRING
- Uso de FUNCTION TRIM
- Controle de vírgula entre objetos JSON
- Totalizadores e SYSOUT
- Encerramento com GOBACK

11. Declaração Sugerida

```

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.

```

```

        SELECT BHCF012S
            ASSIGN TO "BHCF012S.txt"
            ORGANIZATION IS LINE SEQUENTIAL
            FILE STATUS IS GDA-FS-BHCF012S.

```

```

        SELECT BHCF016J
            ASSIGN TO "BHCF016J.json"
            ORGANIZATION IS LINE SEQUENTIAL
            FILE STATUS IS GDA-FS-BHCF016J.

```

```

DATA DIVISION.
FILE SECTION.

```

```

FD  BHCF012S
    RECORDING MODE IS F
    RECORD CONTAINS 65 CHARACTERS.
01  REG-BHCF012S                PIC X(065) .

```

```

FD  BHCF016J.
01  REG-BHCF016J                PIC X(200) .

```

12. Áreas de Controle Sugeridas

```

LOCAL-STORAGE SECTION.

```

```

01  GDA-FILE-STATUS.
    03  GDA-FS-BHCF012S          PIC X(002) VALUE SPACES.
    03  GDA-FS-BHCF016J          PIC X(002) VALUE SPACES.

```

```

03  GDA-FS-OK          PIC X(002) VALUE '00'.

01  WS-FIM-ARQUIVO      PIC X(001) VALUE 'N'.
    88  WS-FIM-SIM      VALUE 'S'.
    88  WS-FIM-NAO      VALUE 'N'.

01  GDA-TOTALIZADORES.
    03  GDA-TOT-LIDOS   PIC 9(005) VALUE ZEROS.
    03  GDA-TOT-JSON    PIC 9(005) VALUE ZEROS.
    03  GDA-TOT-ERROS   PIC 9(005) VALUE ZEROS.

01  GDA-DATA-HORA.
    03  GDA-CURRENT-DATE PIC X(021).

01  GDA-GR-BHCF012S.
    03  FD-CODIGO       PIC X(005).
    03  FD-NOME         PIC X(030).
    03  FD-UF           PIC X(002).
    03  FD-TRILHA       PIC X(010).
    03  FD-SITUACAO     PIC X(010).
    03  FD-DATA         PIC X(008).

```

13. Estrutura da PROCEDURE DIVISION

```

000000-ROTINA-PRINCIPAL
100000-INICIALIZAR
200000-ABRIR-ARQUIVOS
300000-GRAVAR-CABECALHO-JSON
400000-LER-ENTRADA
500000-PROCESSAR-ARQUIVO
510000-GRAVAR-OBJETO-JSON
800000-GRAVAR-RODAPE-JSON
900000-FECHAR-ARQUIVOS
910000-EXIBIR-TOTALIZADORES
999999-FINALIZAR

```

14. Lógica para Controle de Vírgulas

Para evitar vírgula após o último objeto, uma técnica simples é gravar a vírgula antes de cada objeto a partir do segundo registro.

```

IF GDA-TOT-LIDOS > 1
    MOVE "," TO REG-BHCF016J
    WRITE REG-BHCF016J
END-IF

```

15. Pseudocódigo do Processamento

Abrir BHCF012S como entrada
Abrir BHCF016J como saída

Gravar:

```
{
    "participantes": [
```

Ler primeiro registro do BHCF012S

```
Enquanto não for fim de arquivo:
    Somar total de lidos
    Se não for o primeiro objeto:
        Gravar vírgula
    Montar objeto JSON do participante
    Gravar objeto no BHCF016J
    Somar total de objetos JSON
    Ler próximo registro
```

```
Gravar:
    ]
}
```

```
Fechar arquivos
Exibir totalizadores
GOBACK
```

16. Resultado JSON Esperado

```
{
  "participantes": [
    {
      "codigo": "00001",
      "nome": "ANA SILVA",
      "uf": "SP",
      "trilha": "COBOL",
      "situacao": "ATIVO",
      "data": "20260701"
    },
    {
      "codigo": "00002",
      "nome": "BRUNO LIMA",
      "uf": "RJ",
      "trilha": "COBOL",
      "situacao": "ATIVO",
      "data": "20260701"
    },
    {
      "codigo": "00003",
      "nome": "",
      "uf": "MG",
      "trilha": "JCL",
      "situacao": "ATIVO",
      "data": "20260701"
    },
    {
      "codigo": "00005",
      "nome": "ELISA ROCHA",
      "uf": "",
      "trilha": "DB2",
      "situacao": "ATIVO",
      "data": "20260701"
    }
  ]
}
```

Observação: o exemplo acima está reduzido para visualização. O arquivo final deverá conter os 10 participantes.

17. SYSOUT Esperado

```
BHCP0016 - INICIO DO PROCESSAMENTO
DATA/HORA: 202607010547150000000
ARQUIVO BHCF012S ABERTO COM SUCESSO
ARQUIVO BHCF016J ABERTO COM SUCESSO
TOTAL DE REGISTROS LIDOS.....: 00010
TOTAL DE OBJETOS JSON GERADOS.: 00010
TOTAL DE ERROS DE ARQUIVO.....: 00000
BHCP0016 - FIM DO PROCESSAMENTO
DATA/HORA: 202607010547160000000
```

18. Enunciado Oficial para o Aluno

Desenvolver o programa COBOL Batch BHCP0016 para ler o arquivo sequencial BHCF012S, gerado no Exercício 12, e gerar o arquivo BHCF016J.json em formato JSON.

O JSON deverá conter um array chamado participantes com todos os registros lidos. O programa deverá montar manualmente a estrutura JSON utilizando comandos tradicionais de COBOL.

19. Regras do Exercício

1. Abrir BHCF012S como entrada.
2. Abrir BHCF016J como saída.
3. Gravar o cabeçalho do JSON com o objeto principal e o array participantes.
4. Ler todos os registros do arquivo BHCF012S.
5. Gerar um objeto JSON para cada participante.
6. Gravar todos os 10 participantes, inclusive os que possuem campos em branco.
7. Representar campos em branco como string vazia no JSON.
8. Controlar corretamente as vírgulas entre objetos.
9. Gravar o rodapé do JSON.
10. Validar FILE STATUS nas operações de arquivo.
11. Exibir totalizadores no SYSOUT.
12. Encerrar com GOBACK.

20. Critérios de Avaliação

- Programa compila sem erros.
- Arquivo BHCF012S é lido corretamente.
- Arquivo BHCF016J.json é gerado.
- JSON contém objeto principal e array participantes.
- 10 objetos JSON são gerados.
- Registros 00003 e 00005 aparecem com campos vazios como "".
- Não existe vírgula após o último objeto do array.
- FILE STATUS é validado.
- Totalizadores são exibidos corretamente.
- Programa encerra com GOBACK.

21. Checklist Final

- ENVIRONMENT DIVISION com SELECT dos dois arquivos.
- FILE SECTION com layout do BHCF012S.
- Arquivo JSON definido como LINE SEQUENTIAL.

- OPEN INPUT e OPEN OUTPUT implementados.
- READ com AT END implementado.
- Cabeçalho JSON gravado.
- Objetos JSON gravados para todos os participantes.
- Vírgulas entre objetos controladas.
- Rodapé JSON gravado.
- CLOSE dos arquivos implementado.
- SYSOUT com totalizadores implementado.